

Description of Simulator $\mathcal{S}(1^\lambda, \mathbb{X})$:

Compute commitment $c = \text{Commit}(0^{|w_{\text{R1CS}}|+m}, u)$ and $c' = \text{Commit}(0^{|w_{\text{R1CS}}|+m}, u')$.

Sample a random vector $\mathbf{r} \leftarrow [0, 2^\kappa)^n$, and define the instance $\mathbb{X}'_{\text{R1CS}}$ and $\mathbb{X}^*_{\text{R1CS}}$ accordingly.

Run the simulator $\mathcal{S}_{\text{R1CS}}$ on input $(c + c', \mathbb{X}'_{\text{R1CS}})$ and $(c', \mathbb{X}^*_{\text{R1CS}})$. Obtain proofs π' and π^* .

Output $(\mathbf{r}, c, c', \pi', \pi^*)$.

Indistinguishability is proven using the following hybrids:

1. Instead of honestly generating the transcripts of the underlying commit-and-prove protocol, we simulate these transcripts using $\mathcal{S}_{\text{R1CS}}$: $\pi' \leftarrow \mathcal{S}_{\text{R1CS}}(c + c', \mathbb{X}'_{\text{R1CS}})$; $\pi^* \leftarrow \mathcal{S}_{\text{R1CS}}(c', \mathbb{X}^*_{\text{R1CS}})$. Indistinguishability follows from the zero-knowledge property of the underlying commit-and-prove protocol.
2. Instead of generating the commitments c and c' honestly using the witness w_{R1CS} , we commit to all 0 strings for both commitments. Indistinguishability follows from the hiding property of commitment schemes.

5.6.7 Proof of Soundness with Zero-Knowledge

We first use the emulator $\mathcal{E}$ to extract the message committed in c and c' . Then we define the event:

BAD : $\{\exists i : v_i \notin [n \cdot 2^{w_i+\kappa}, n \cdot 2^{w_i+\kappa}) \cup (\text{prefix of } c') \neq 0^{|w_{\text{R1CS}}|}\}$. First notice that if the event **BAD** does not happen, then the commitment $c + c'$ indeed commits to some R1CS witness vector w_{R1CS} due to the homomorphic commitment. Furthermore, since for each $i \in [l]$, $v_i \in [n \cdot 2^{w_i+\kappa}, n \cdot 2^{w_i+\kappa})$, by our analysis in lemma 5.10, the soundness error of our commit-and-prove protocol given in Fig 5.3 is at most ϵ . Moreover, due to the soundness of underlying commit-and-prove protocol, $\Pr[\mathbf{BAD} : \mathcal{V}(1^\lambda, \mathbb{X}'_{\text{R1CS}}) = 1 \wedge \mathcal{V}(1^\lambda, \mathbb{X}^*_{\text{R1CS}}) = 1] \leq \epsilon'$ for some soundness error ϵ' . Therefore, our total soundness error (with zero-knowledge) is at most $\epsilon + \epsilon'$.

Obtaining Full-fledged ZK-SNARG Finally, via the Fiat-Shamir transformation, we obtain a ZK-SNARG for floating/fixed point computations.

5.6.8 Extension to Floating Point Computation

Recall that a *floating point number* of precision w is a pair of integers (s, e) , which represent a real number $s \cdot 2^{e-w}$, where $|s| \in [2^{w-1}, 2^w)$ is an integer represented in $w+1$ bits with 1 sign bit and w bit fraction (called the significand) where the fraction is always normalized such that the most significant bit is fixed to 1. We first present a high-level overview of our R1CS compiler which compiles any floating point computation into a R1CS instance. Starting from this point, we make two inherent relaxations for the ease of our compiler construction. Firstly, we fix $\delta = 2^{-w}$ for addition/subtraction gate and $\delta = 2^{-2w}$ for multiplication gate. Secondly, we relax all the constraints (as defined in 5.2) from strict inequalities to inequalities.

Adding/Subtracting Two Floating Point Numbers. Suppose we want to add/subtract two floating point numbers: $s_a \cdot 2^{e_a} \cdot w$ and $s_b \cdot 2^{e_b} \cdot w$. Let the outcome of such addition/subtraction be $s_c \cdot 2^{e_c} \cdot w$. Following δ -approximate correctness, we want to ensure that:

$$|s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} - s_c \cdot 2^{e_c}| \leq \delta \cdot |s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b}|,$$

where we have multiplied both sides by 2^w to simplify the inequality. We also incorporate subtraction into addition by allowing each summand to take negative values.

A Naive Approach. Checking floating point numbers addition is particularly challenging due to the fact that we also need to take care of their exponents. To see why, consider a naive (however very inefficient) way to convert this inequality into R1CS constraints: Let's first define intermediate variables and constraints that mimic all exponentiations: (e.g. we can define $\lambda_a := 2^{e_a}$ and so on). Once we have these intermediate exponentiations, one can then check:

$$|s_a \cdot \lambda_a + s_b \cdot \lambda_b - s_c \cdot \lambda_c| \leq \delta \cdot |s_a \cdot \lambda_a + s_b \cdot \lambda_b|.$$

If we take the upper bound of the size of each summand, we can essentially view each summand as a (very large) fixed-point number. In this way we can apply our previous range proof technique to this inequality just like the case of fixed point addition.

This naive approach mainly suffers from two sources of inefficiency: Firstly, the best known method to check exponentiation requires bit decomposing the exponent, and then use repeated squaring algorithm to break down the exponentiation procedure into one multiplication and addition at each step, thus turning them into constraints compatible with R1CS. Clearly, we need to bit decompose all of e_a , e_b and e_c , turning them into roughly $3 \times (2k + 1)$ constraints with $3 \times (2k)$ new variables. We reduce the number of constraints and variables by roughly a factor of 3, via only one bit decomposition and some extra constraints. For large value of k (for example, $k = 11$ in a 64-bit floating point number) this will greatly increase the prover time. Secondly, for large k , the value $\lambda_a = 2^{e_a} \approx 2^{2^k}$ will typically become very large. This will cause the upper bound on each summand to significantly grow, which pushes the choice of the modulus p to become large (for example, in a 32-bit floating point number, $2^k < 128$ and we need to choose p to be around 670 bits). In our approach, we reduce this upper bound so that it no longer depends on k , but on $\log(w)$ instead. This will allow us to choose a smaller p to again reduce prover computation (in the last example, $\log(w) \approx 5$ so we can choose p to be around 270 bits).

Converting Addition into Constraints: Our Approach. We begin explaining our idea with a simple yet useful observation:

Claim 5.19. For $|s_a|, |s_b|, |s_c| \in [2^{w-1}, 2^w)$ and $\delta = 2^{-w}$, if $|s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} - s_c \cdot 2^{e_c}| \leq \delta \cdot |s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b}|$, then $\min(e_a - e_b, e_b - e_c, e_a - e_c) \in \{-1, 0, 1\}$.

Proof. First suppose that among s_a , s_b and s_c , two of them have the same signs and the other has the opposite sign, we claim that this is impossible: by renaming the variables, we can always assume that s_a and s_b have the same signs. Furthermore, since we can always

negate the signs of all variables within an absolute clause, let's always assume that $s_a, s_b \geq 0$, such that $s_c \geq 0$. Therefore,

$$\begin{aligned} & |s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} - s_c \cdot 2^{e_c}| \\ &= s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} - s_c \cdot 2^{e_c} \\ &\geq s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b}. \end{aligned}$$

Since $\delta \cdot s_a < s_a$ and $\delta \cdot s_b < s_b$, $s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} > \delta \cdot |s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b}|$, which is a contradiction. Now suppose that all of s_a, s_b and s_c have the same (positive) signs, and furthermore assume $s_a \cdot 2^{e_a} \geq s_b \cdot 2^{e_b}$ by renaming the variables if necessary, then we have: $|s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} - s_c \cdot 2^{e_c}| \leq \delta \cdot (s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b})$,

$$(1 - \delta)s_a \cdot 2^{e_a} + (1 - \delta)s_b \cdot 2^{e_b} \leq s_c \cdot 2^{e_c} \leq (1 + \delta)s_a \cdot 2^{e_a} + (1 + \delta)s_b \cdot 2^{e_b}.$$

Since $\delta = 2^{-w}$, and $s_a \geq 2^{w-1}$, $(1 - \delta)s_a \approx s_a \approx (1 + \delta)s_a$ and $(1 - \delta)s_b \approx s_b \approx (1 + \delta)s_b$, so that we have: $s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} \approx s_c \cdot 2^{e_c}$. Since $s_a \cdot 2^{e_a} \geq s_b \cdot 2^{e_b}$, then: $s_c \cdot 2^{e_c} \approx s_a \cdot 2^{e_a} + s_b \cdot 2^{e_b} \leq s_a \cdot 2^{e_a+1}$. Given that both s_a, s_c are normalized, by comparing the terms, we get $e_c \leq e_a + 1$, hence $\min(e_a - e_b, e_b - e_c, e_a - e_c) \in \{-1, 0, 1\}$. $\square$

To express each floating point number addition into a number of R1CS constraints, let's utilize the above observation by dividing 2^{e_c} from both sides of the inequality and also defining $\theta_a := e_a - e_c$ and $\theta_b := e_b - e_c$. Notice that we have three possible scenarios:

1. **Case I:** $\theta_a \in \{-1, 0, 1\} \Leftrightarrow e_a - e_c \in \{-1, 0, 1\}$.
2. **Case II:** $\theta_b \in \{-1, 0, 1\} \Leftrightarrow e_b - e_c \in \{-1, 0, 1\}$.
3. **Case III:** $\theta_a - \theta_b \in \{-1, 0, 1\} \Leftrightarrow e_a - e_b \in \{-1, 0, 1\}$.

Meanwhile, the inequality translates into: $|s_a \cdot 2^{\theta_a} + s_b \cdot 2^{\theta_b} - s_c| \leq \delta \cdot |s_a \cdot 2^{\theta_a} + s_b \cdot 2^{\theta_b}|$.

Our goal is to use only one bit decomposition to enforce two constraints: ($\lambda_a = 2^{\theta_a}$ and $\lambda_b = 2^{\theta_b}$). In order to achieve this goal, observe that in each of the three possible scenarios, we only need to enforce one such constraint (hence one bit decomposition): $\lambda_\theta = 2^\theta$, where θ is either θ_a or θ_b . The reason is as follows:

1. **Case I:** Since θ_a is small, we can rewrite its constraint using Lagrange interpolation as: $\lambda_{\theta_a} = \frac{\theta_a(\theta_a - 1)}{4} + \theta_a(\theta_a + 1) - (\theta_a + 1)(\theta_a - 1)$. Now set $\theta = \theta_b$, and add the additional constraint that $\lambda_\theta = 2^\theta$.
2. **Case II:** This case resembles a similar treatment as Case I, where $\theta = \theta_a$.
3. **Case III:** Let's set $\theta = \theta_b$, and add the constraint that $\lambda_\theta = 2^\theta$. Now define $\theta_{a,b} = \theta_a - \theta_b$, which is again small, meaning that we can again use Lagrange interpolation to derive the constraint for θ_a as:

$$\lambda_{\theta_a}^* = \frac{\theta_{a,b}(\theta_{a,b} - 1)}{4} \cdot \lambda_\theta + \theta_{a,b}(\theta_{a,b} + 1) \cdot \lambda_\theta - (\theta_{a,b} + 1)(\theta_{a,b} - 1) \cdot \lambda_\theta.$$

In order to determine whether $\theta = \theta_a$ or $\theta = \theta_b$, we use three relaxed indicator variables and enforce each of their values to take either 0 or 1:

1. $1_{\theta_a}(1 - 1_{\theta_a}) = 0$, and $1_{\theta_a} = 1 \implies$ Case I happens (details later);
2. $1_{\theta_b}(1 - 1_{\theta_b}) = 0$, and $1_{\theta_b} = 1 \implies$ Case II happens;
3. $1_{\neg a,b}(1 - 1_{\neg a,b}) = 0$, and $1_{\neg a,b} = 1 \implies$ Case III happens.

Notice that we do not require the reverse of these statements to be true. Now we add two more constraints:

1. $1_{\theta_a} + 1_{\theta_b} + 1_{\neg a,b} = 1$;
2. $\theta = 1_{\theta_a} \cdot \theta_b + 1_{\theta_b} \cdot \theta_a + 1_{\neg a,b} \cdot \theta_b$.

These constraints will enforce θ to take the value of either θ_a or θ_b , depending on the desired scenario.

Turn Inequality into Equality for Large θ . A subtle issue in the above construction is that if the absolute value of θ becomes large, the modulus p needs also to grow large for soundness to hold. To understand this issue, let's take case I as an example: we want to use range proof to show the following inequality:

$$|s_a \cdot 2^{\theta_a} + s_b \cdot 2^{\theta} - s_c| \leq \delta \cdot |s_a \cdot 2^{\theta_a} + s_b \cdot 2^{\theta}|.$$

Notice that both θ_a and θ may be negative (thus leaving fractional numbers on both sides), and we are checking this inequality over mod p . To ensure the inequality also holds over the reals, we return to our previous techniques: First let's multiply both sides by $2^{|\theta|+1}$ to ensure that all of the exponents are positive (thus removing fractional numbers):

$$\begin{aligned} & |s_a \cdot 2^{\theta_a+|\theta|+1} + s_b \cdot 2^{\theta+|\theta|+1} - s_c \cdot 2^{|\theta|+1}| \\ & \leq \delta \cdot |s_a \cdot 2^{\theta_a+|\theta|+1} + s_b \cdot 2^{\theta+|\theta|+1}|. \end{aligned}$$

Now if we choose the bit length of modulus p to be larger than $2 \log(n) + 4w + 3\kappa + 12 + 2^2 \cdot |\theta|$, the above inequality will hold over the reals. Clearly, the bit length of p needs to grow with $2^{2|\theta|} \approx 2^{k+1}$.

The increased group size will induce a significant cost on prover computation. Fortunately, we observe that when $|\theta|$ exceeds $w + 2$, we can get rid of the dependence on $|\theta|$ by substituting all inequalities with different equalities for each of the aforementioned three scenarios:

1. **Case I:** There are two possibilities:

- (a) $\theta > w+2$: Since s_a, s_b, s_c are normalized and $|\theta_a| \leq 1$, we can bound $|s_a \cdot 2^{\theta_a} - s_c| < 2^2 \cdot |s_b|$. Therefore, $|s_a \cdot 2^{\theta_a} + s_b \cdot 2^{\theta} - s_c| > |s_b \cdot 2^w| > \delta \cdot |s_a \cdot 2^{\theta_a} + s_b \cdot 2^{\theta}|$. Thus we always reject.

- (b) $\theta < (w+2)$: In this case $|s_b \cdot 2^\theta|$ is relatively small compared to $|s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta|$. Our intuition is that one can simply drop the summand $|s_b \cdot 2^\theta|$ while still obeying the inequality. More formally, since $|s_b \cdot 2^\theta| < |s_b \cdot 2^{w-2}| < |s_a \cdot 2^{w-1}| < \delta \cdot |s_a \cdot 2^{\theta_a}| \leq \delta \cdot |s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta|$, we can instead enforce the equality constraint that $s_a \cdot 2^{\theta_a} = s_c$. This is done by checking $s_a = s_c$ and $e_a = e_c$.

2. **Case II:** This case resembles the analysis of Case I.

3. **Case III:** There are two possibilities:

- (a) $s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta = 0$: In this case, we simply check if $s_c = 0$ (In our construction we ignore this case for simplicity).
- (b) Otherwise, first assume $\theta > w+2$, notice that $|s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta| \geq 2^\theta |s_b + s_a/2| \geq 2^{\theta-1}$. Furthermore, $|s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta| \leq 2^\theta |s_b + 2s_a| \leq 2^{\theta+2}$. As a result, we deduce that:

$$\begin{aligned} & |s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta - s_c| \\ & \geq |s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta| - |s_c| \\ & \geq 2^{\theta-1} - |s_c| \geq 2^{\theta-1} - 2^w, \end{aligned}$$

on the other hand, $\delta \cdot |s_a \cdot 2^{\theta_a} + s_b \cdot 2^\theta| \leq 2^{\theta+2-w}$.

For $w > 2$ and $\theta > w+2$, one can observe that $2^{\theta+2-w} < 2^{\theta-1} - 2^w$, thus the desired inequality will not hold. The other scenario where $\theta < (w+2)$ is very similar. Therefore we should always reject when $|\theta| > w+2$.

To conclude, we can choose the bit length of p to grow with $2^2 \cdot \max(\theta, w+2)$ instead of θ . In practice we will simplify our constraints by picking the cutoff to be $w+8 = 2^5$ for 32 bit floating point numbers, or $w+9 = 2^6$ for 64 bit floating point numbers.

Multiplying Two Floating Point Numbers. Suppose we want to multiply two floating point numbers: $s_a \cdot 2^{e_a-w}$ and $s_b \cdot 2^{e_b-w}$. Let the outcome of such multiplication be $s_c \cdot 2^{e_c-w}$. Following δ -approximate correctness 5.2, we want to ensure that:

$$|s_a \cdot 2^{e_a} \cdot s_b \cdot 2^{e_b} - s_c \cdot 2^{e_c}| \leq \delta \cdot |s_a \cdot 2^{e_a} \cdot s_b \cdot 2^{e_b}|.$$

Our methodology for multiplications will align with our previous route for addition. We begin with yet another simpler observation about the relationship between these exponents (e_a, e_b, e_c) .

Claim 5.20. For $|s_a|, |s_b|, |s_c| \in [2^{w-1}, 2^w)$ and $\delta = 2^{-w}$, if $|s_a \cdot 2^{e_a} \cdot s_b \cdot 2^{e_b} - s_c \cdot 2^{e_c}| \leq \delta \cdot |s_a \cdot 2^{e_a} \cdot s_b \cdot 2^{e_b}|$, then $e_c - (e_a + e_b) \in \{w-1, w\}$.

Proof. First, let's rewrite the inequality as:

$|s_a \cdot s_b - s_c \cdot 2^{e_c - (e_a + e_b)}| \leq \delta \cdot |s_a \cdot s_b|$. Since $|s_a|, |s_b| \in [2^{w-1}, 2^w)$, we have $2^{2w-2} \leq |s_a \cdot s_b| < 2^{2w}$. WLOG assume $s_a s_b \geq 0$ so that $s_c \geq 0$ (the other case is similar), then: $s_a s_b (1 - \delta) \leq s_c \cdot 2^{e_c - (e_a + e_b)} \leq s_a s_b (1 + \delta)$.